MICROCONTROLLER INTERNALS

Application: Data driven Vs IO driven

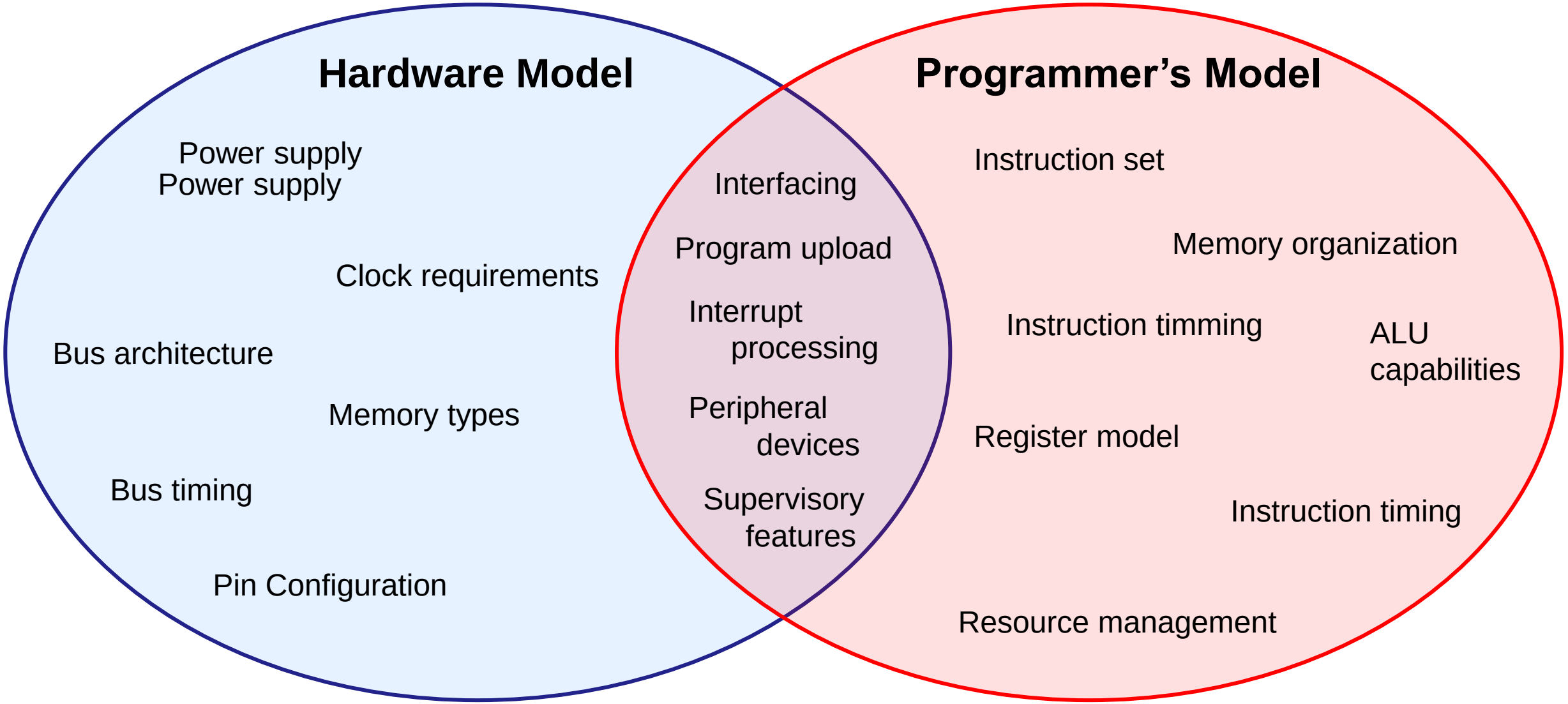
DATA DRIVEN

- *Lots of data-in-memory processing. Memory loaded with data in advance*
- *Large, fast accessible memory space*
- *Complex instruction set supporting different types of number crunching procedures*
- *Synchronous data use: Data is available in memory whenever instructions need them*
- *Internal events: Responses are mostly to internal changes*
- *Ideally suitable for stored-program serial processing*

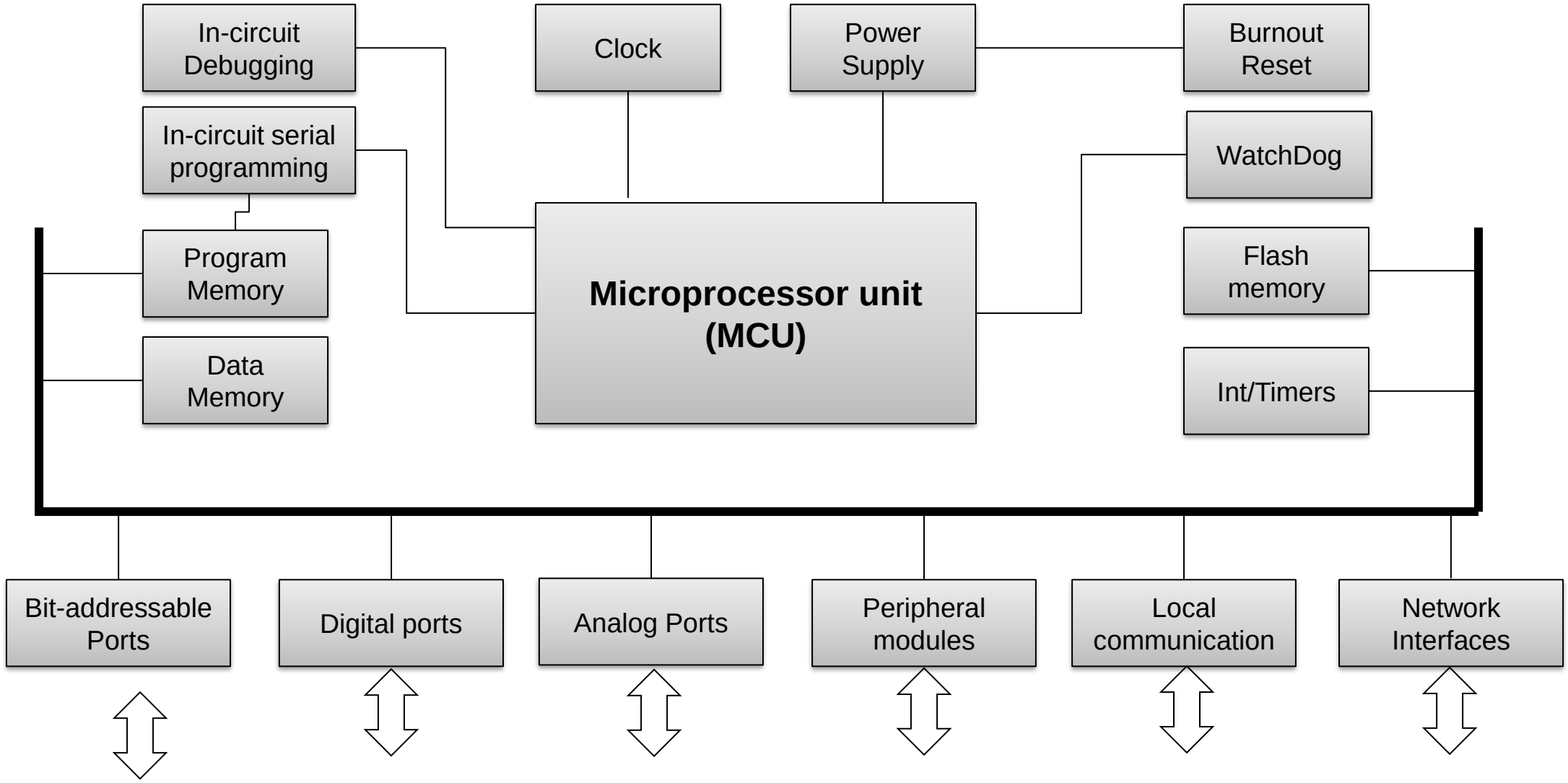
IO DRIVEN

- *Most data is in the environment (i.e. sensors) and available only when a change has occurred*
- *Limited data in memory. Speed and size not critical*
- *Asynchronous data use: Data arrived when the environment reacts, not when the algorithm needs them*
- *Simple, but fast instruction set*
- *Need to provide real-time response to external event.*
- *Need to handle multiple external events at the same time. Most requiring some immediate processing*

Hardware and Programmer's views



Components of a microcontroller



Microprocessor unit (MPU) in Microcontroller

- *Could be a specialized design or a scaled-down version of a regular microprocessor*
  - *E.g. Intel 8048, 8051, Motorolla 68HC11*
- *Follow Harvard Architecture with the separate program and data memory*
  - *Program memory may have a larger word size – allowing long word sizes*
  - *Unified and simple timing – single clock cycle instructions*
  - *A Few RISC instructions*
- *Data memory of often viewed as a "register file" having much faster access compared to bus-based architecture*
- *Often use memory-mapped IO with built-in peripheral registers appearing as memory locations*
  - *Allow IO access like memory locations*
  - *Memory may not be continuous in some designs*
- *Program memory is usually read-only during execution*
  - *Can also be read-protected to safeguard intellectual property*
- *Provided with a separate hardware stack for subroutine calls*

MPU Supervisory Modules

- *Monitor and ensure that the MPU functions correctly*
  - *Watch Dog Timer*
    - *Ensure that software has not hung-up in a endless loop. Causes a master reset after a timeout unless reset by software*
  - *Burnout detect*
    - *Monitor power supply and causes a master reset when a glitch / burnout is detected*
  - *Start-up delay*
    - *Delay processor execution by a pre-determined time at power-on allowing high energy devices to settle down on power requirements*
  - *Oscillator delay*
    - *Delay processor execution by a pre-defined period until the main clock oscillator becomes stable*

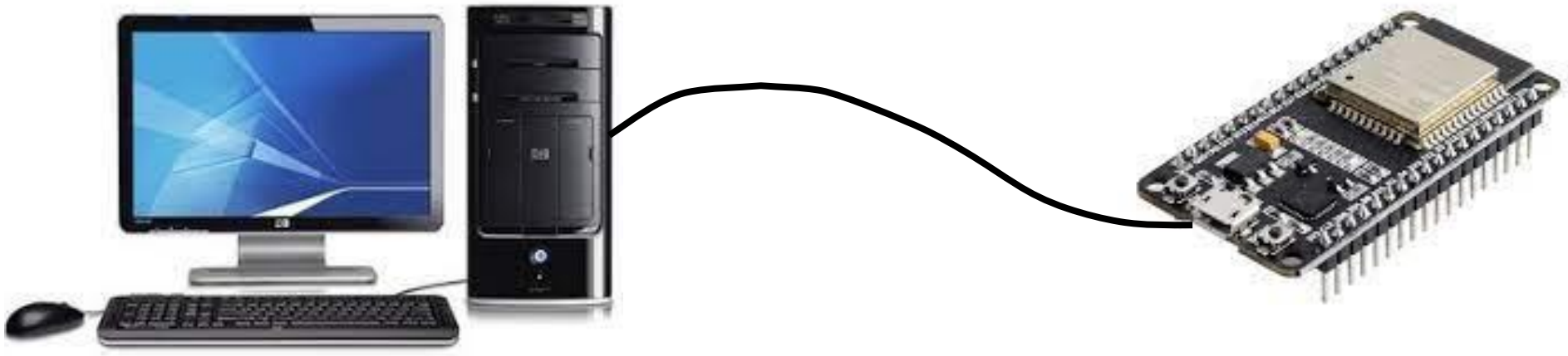
In-Circuit programming & In-Circuit Debugging

ICSP: In Circuit Serial Programming

ICD: In circuit Debugging

Allows program downloading and debugging without any external sophisticated equipment.

- *ICSP writes the object code directly to the program memory.*
- *ICD works with the MPU providing insight into program execution*



Serial link

Host Computer

Microcontroller

In-Circuit programming & In-Circuit Debugging

OTA: Over the Air Update
(Wireless Programming)

Allows the firmware to be uploaded/updated over a remote WiFi connection.

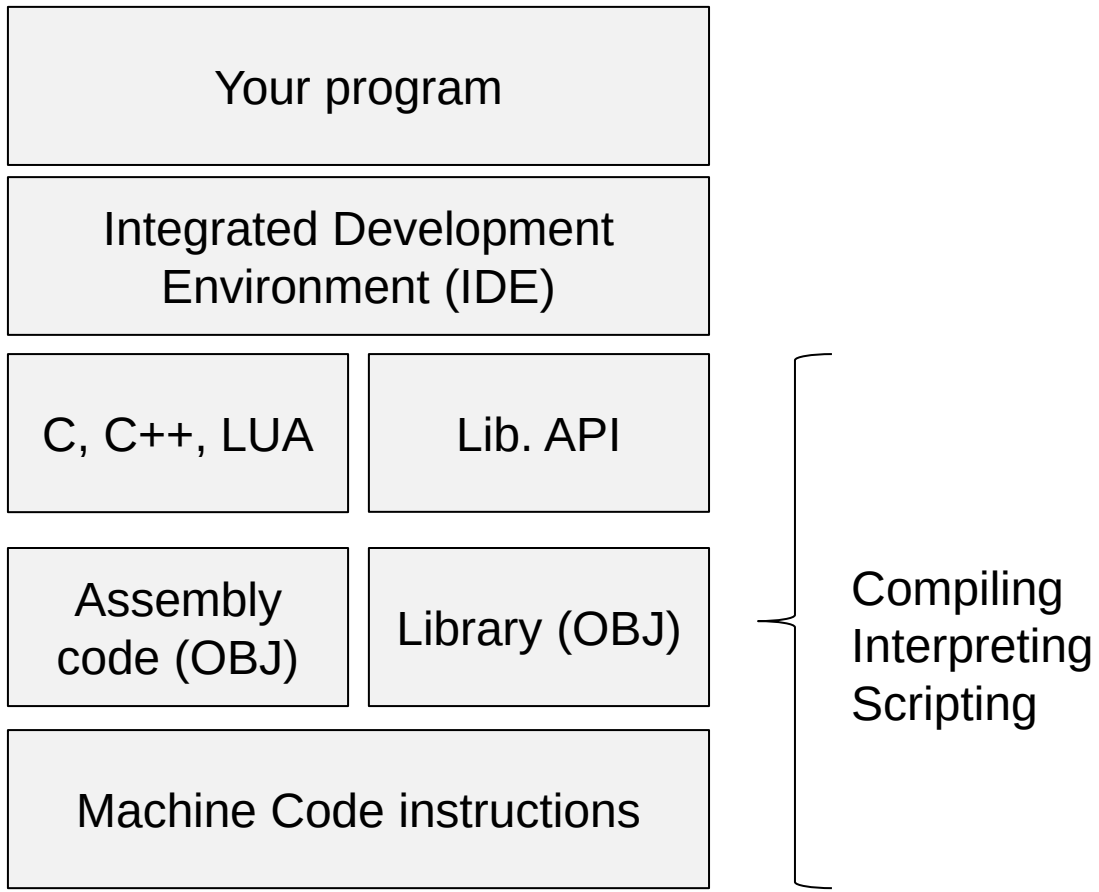
- *Not done at the hardware level, but using a bootstrap software module.*
- *Has limitations on debugging features*

Host Computer

WiFi link

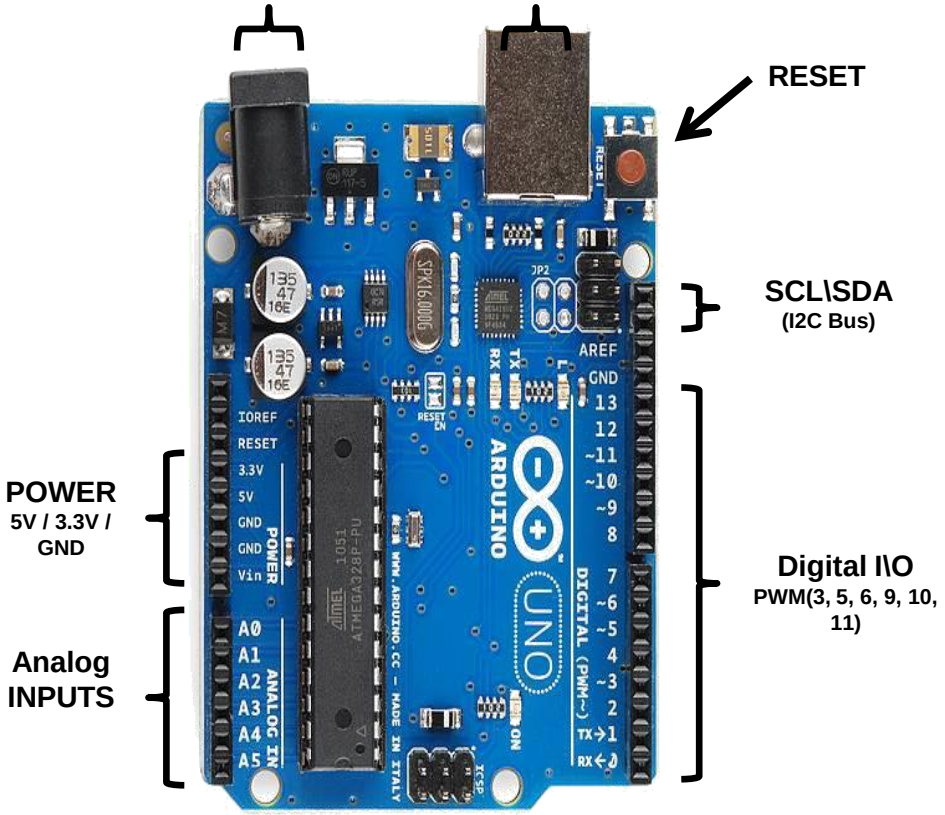
Microcontroller

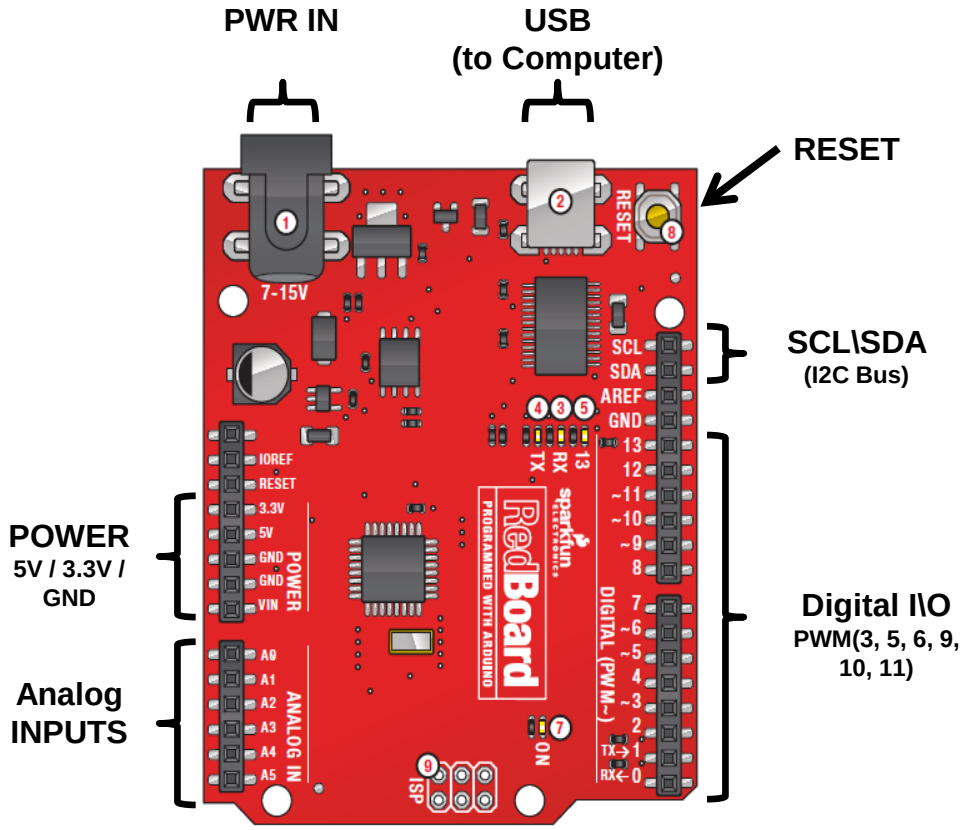
Microcontroller programming



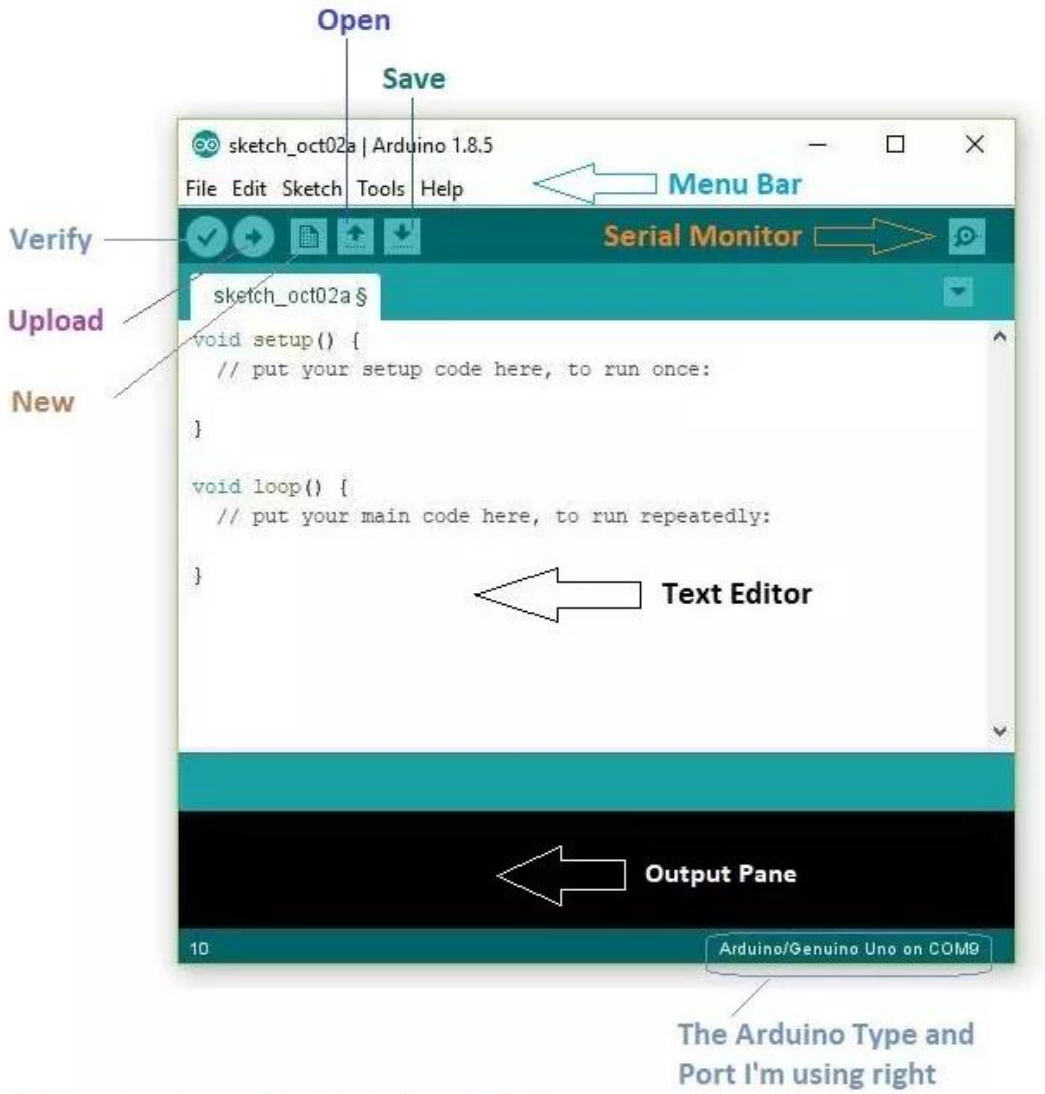
- *What would you need?*
  - *A microcontroller with a development board*
  - *A computer with an IDE installed*
  - *Program download cable/adapter*
  - *A power supply*
  - *Input / Output devices*

Getting Started with Arduino: The UNO board





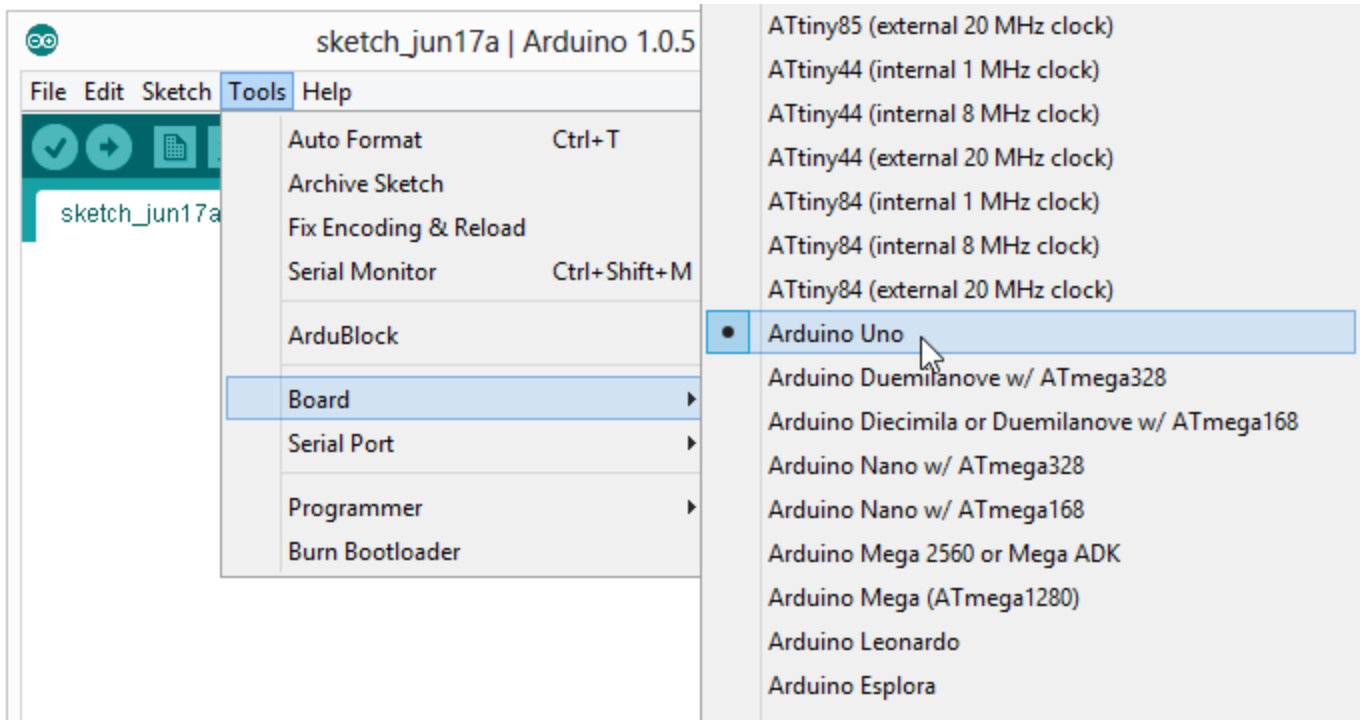
The Arduino IDE software



- *You can download it from*

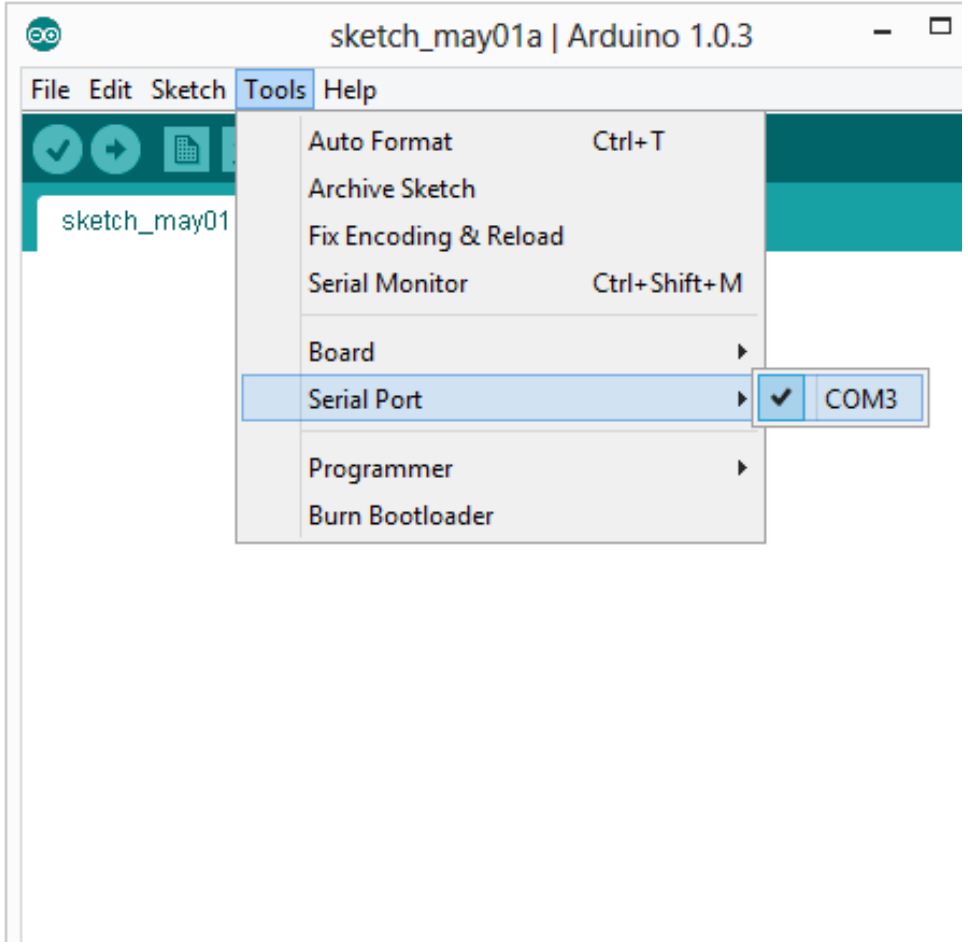
  https://www.arduino.cc/en/software
- *Runs on Windows, Linux and IOS*
- *You may also need to download a USB driver depending on your development board*
  - *Plug the development board to a USB port and see whether a COM port is recognized. If not, download and install the driver*

Setting the boardTools → Board



- *Next, double-check that the proper board is selected under the Tools→Board menu.*

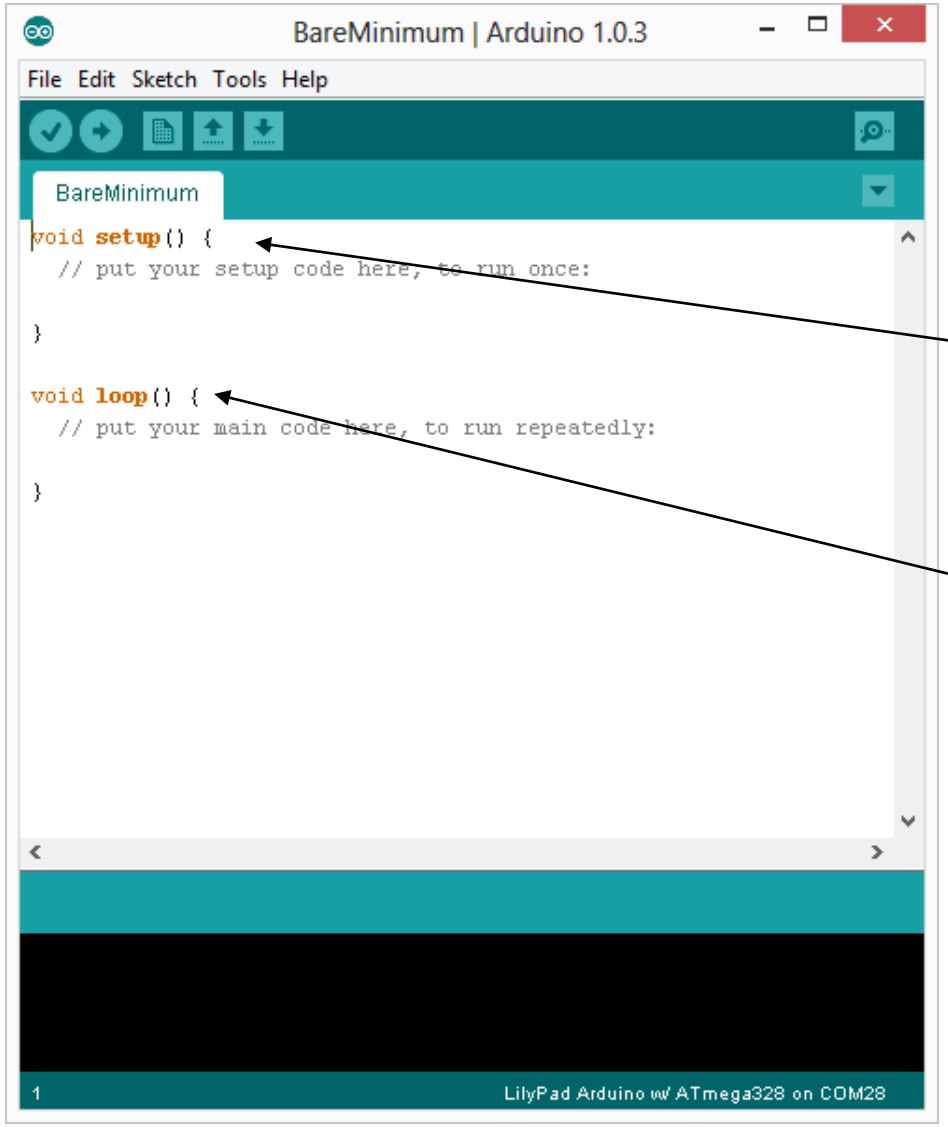
Setting the serial port: Tools → Serial Port



- *Your computer communicates to the Arduino microcontroller via a serial port → through a USB-Serial adapter.*

- *Check to make sure that the drivers are properly installed.*

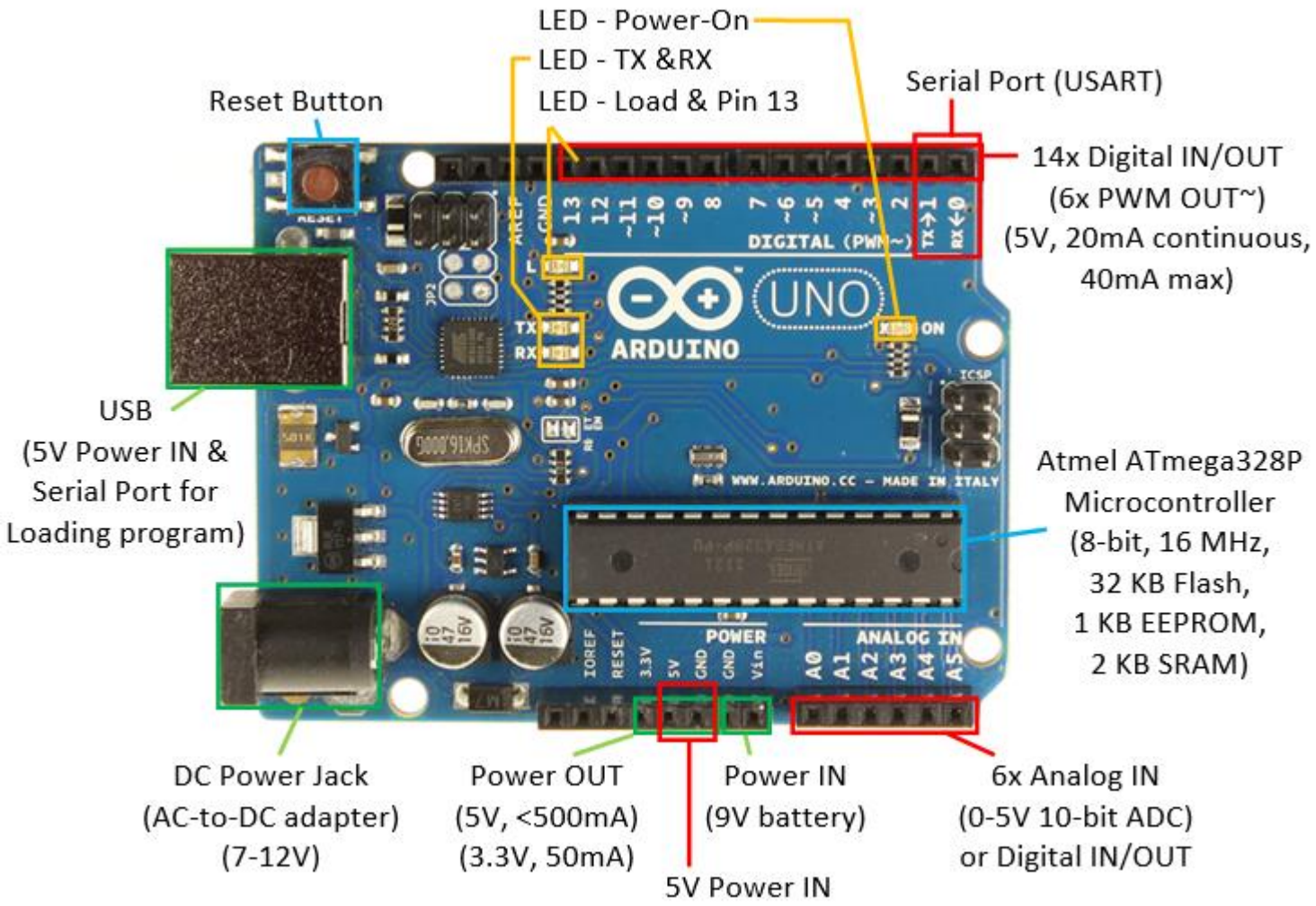
Structure of an Arduino sketch

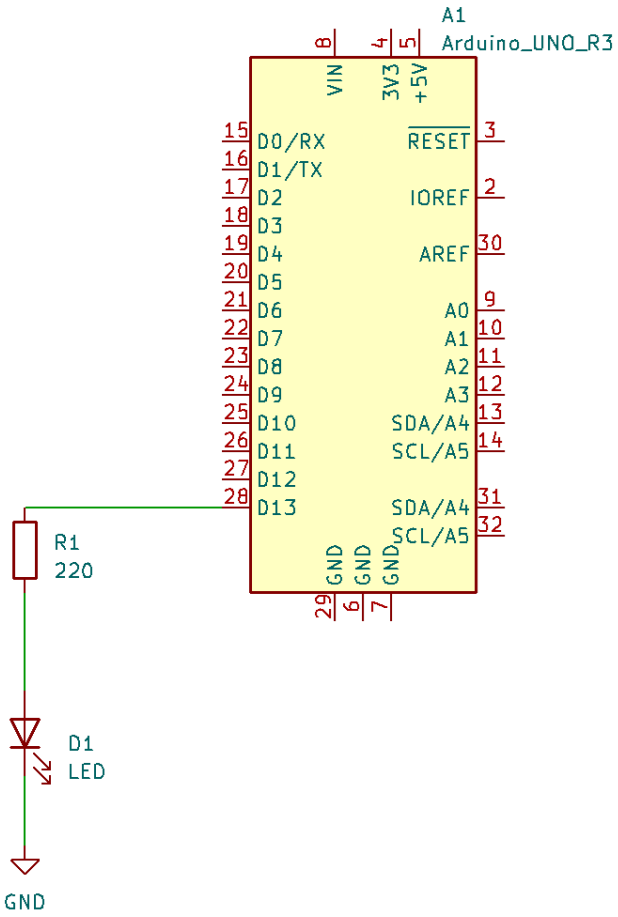


- *Two important functions/methods*

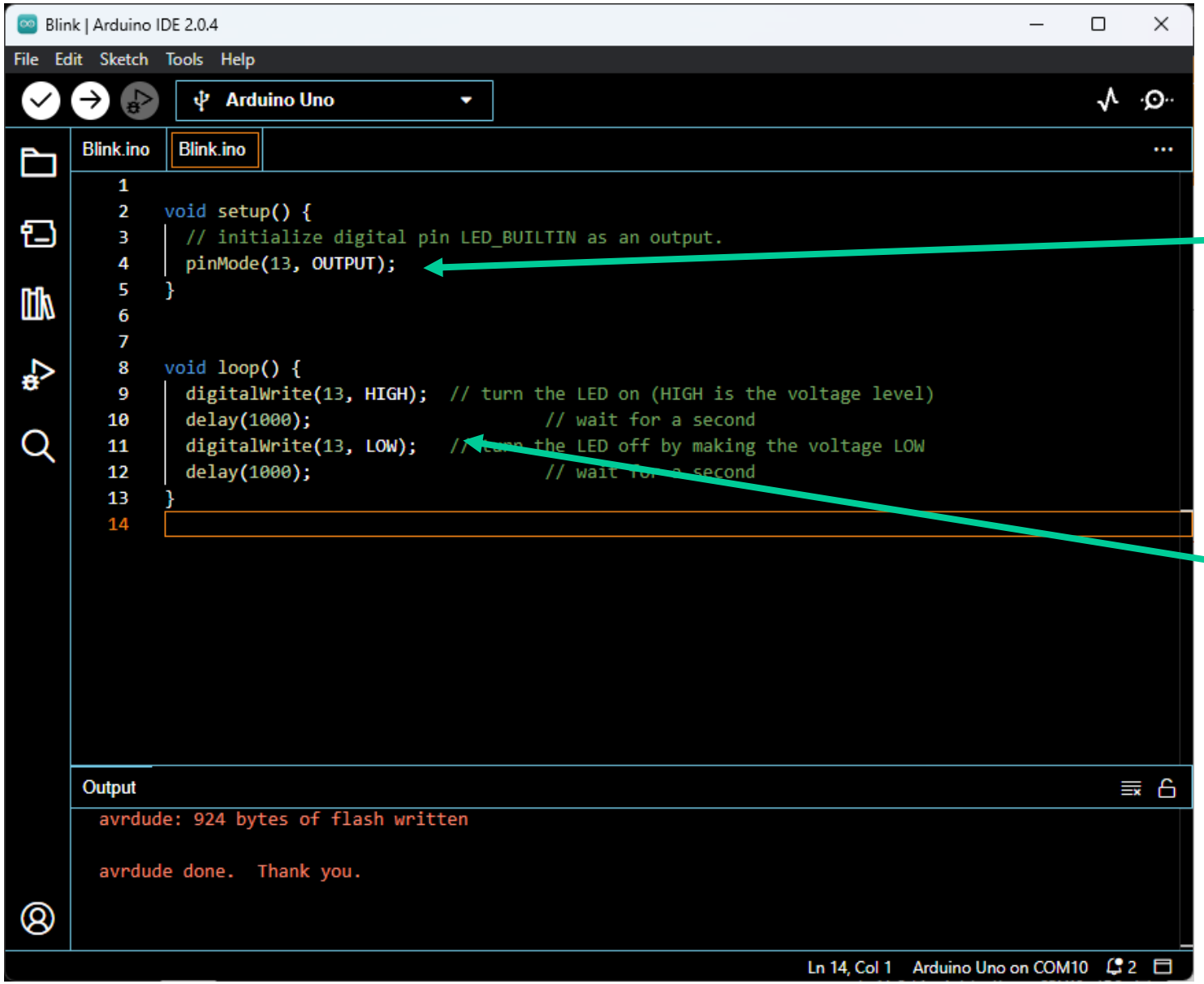
```
void setup()
{
        // runs once
}

void loop()
{
        // repeats
}
```

Blinking the in-built LED





Blinking the in-built LED



By default, all pins are set to input mode. So set pin# D13 to output mode at startup

Loop that executes continuously setting pin# D13 to high/low and waiting for 1000ms (1s) in between

Let's see a live demo!

The microcontroller advantage…

- *Suppose we want to do the same with Z80 microcontroller. Out LED will be connected to bit 0 of a port having address 0x378*

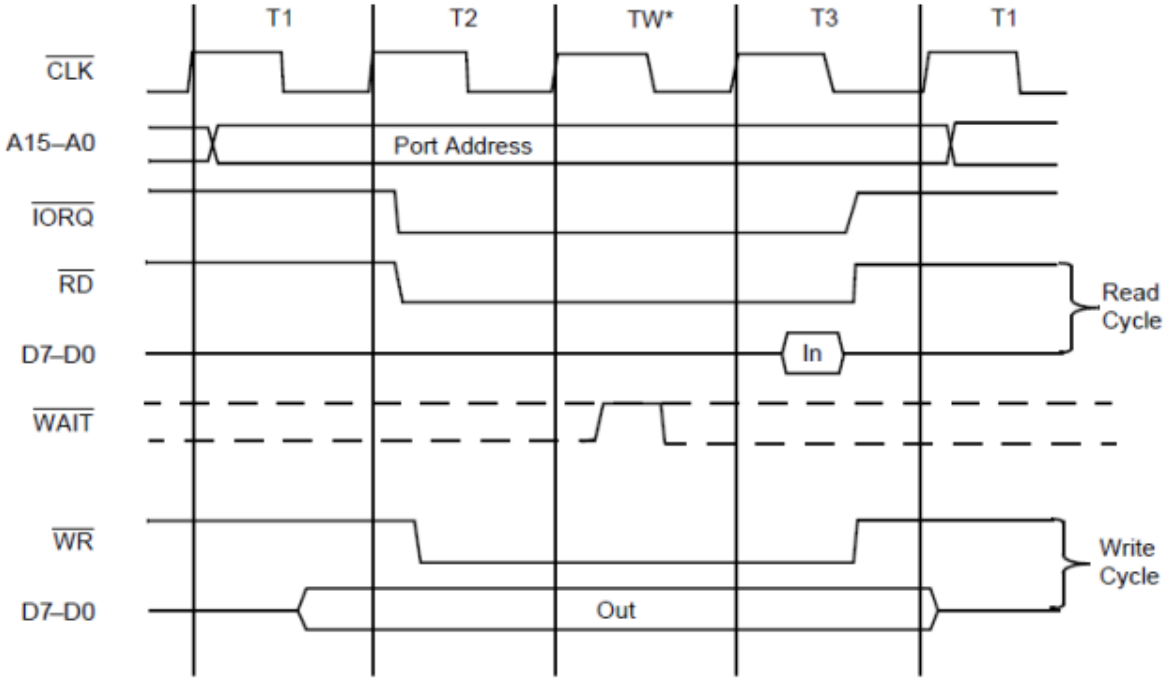
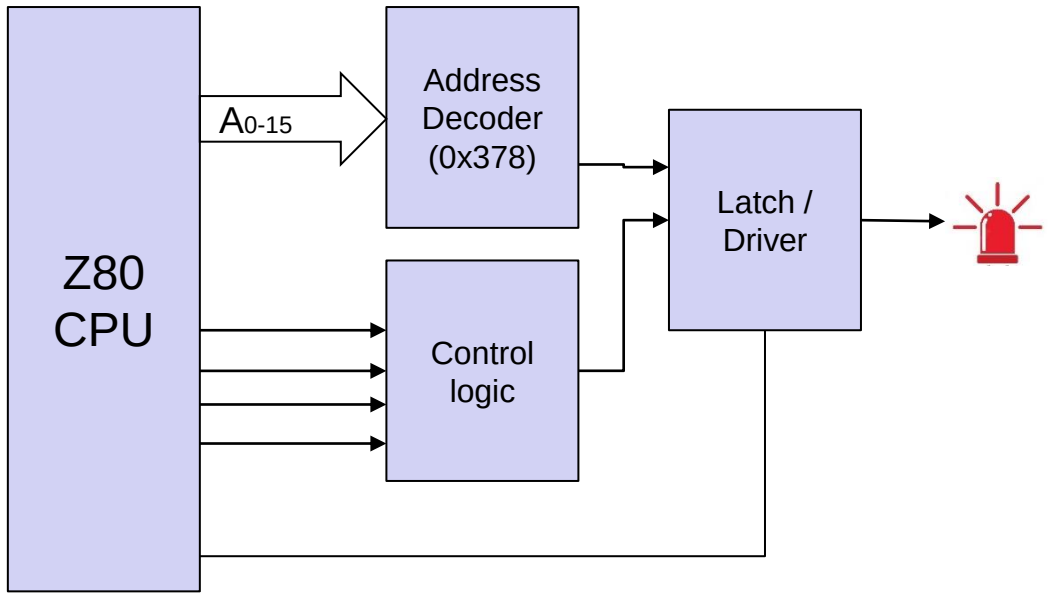
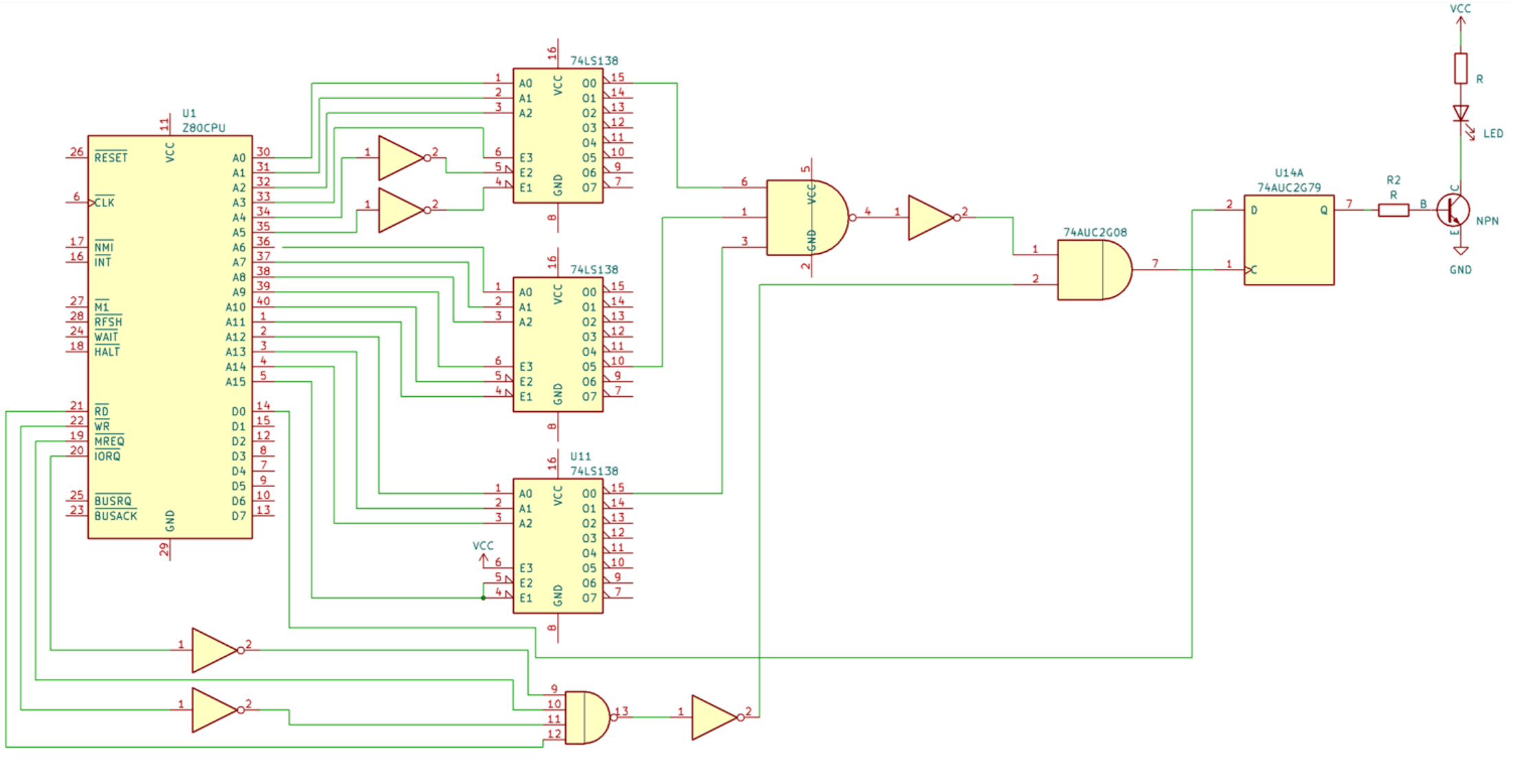


Figure 7. Input or Output Cycles

Note: *In Figure 7, TW is an automatically-inserted WAIT state.



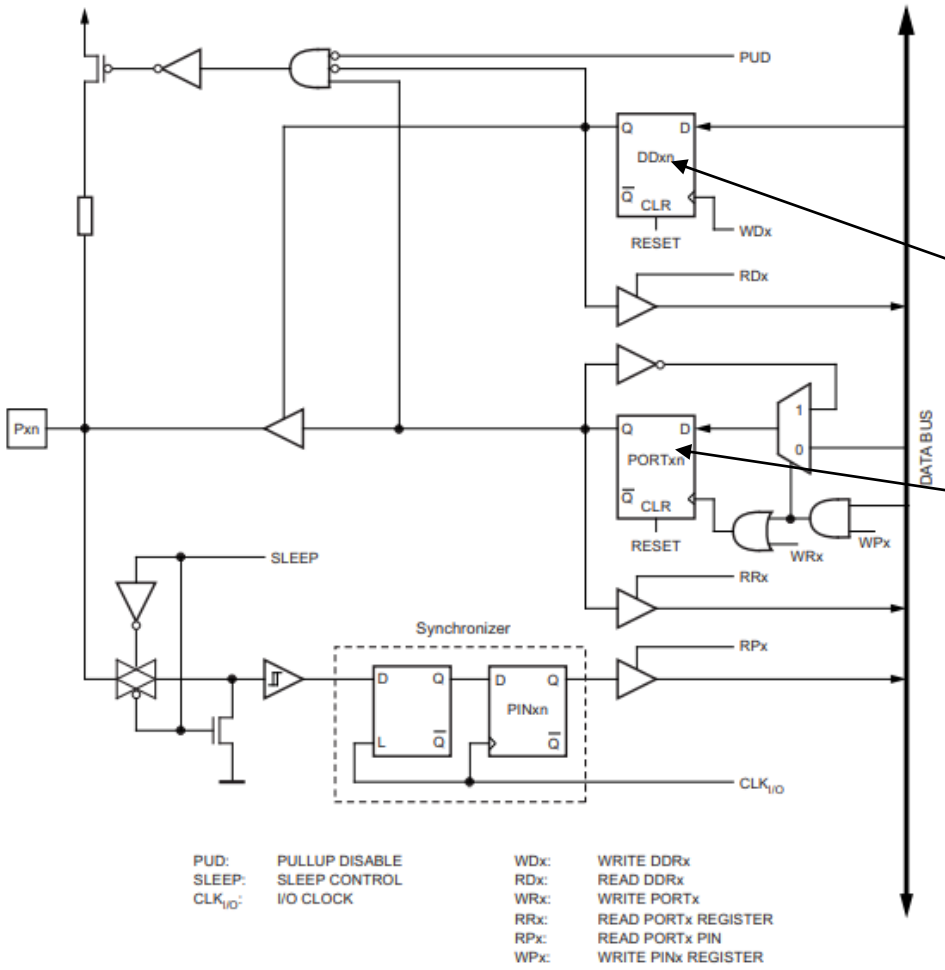
The microcontroller advantage…



Bit Addressable Digital IO Ports

- *Bit addressable IO ports provide direct access to individual pins of the microcontroller via their respective control / data registers*
- *At hardware level these pins can provide different functions and capabilities, often controlled by the configuration register*
  - *Simple digital input / output with data latching*
  - *Combined analog / digital IO*
  - *Additional physical layer properties such as*
    - *Resister Pull-up / Pull-down*
    - *Open collector (drain) with tri-state outputs*
    - *Schematic trigger (hysteresis) inputs*

What happens inside the ATMEGA238 uC

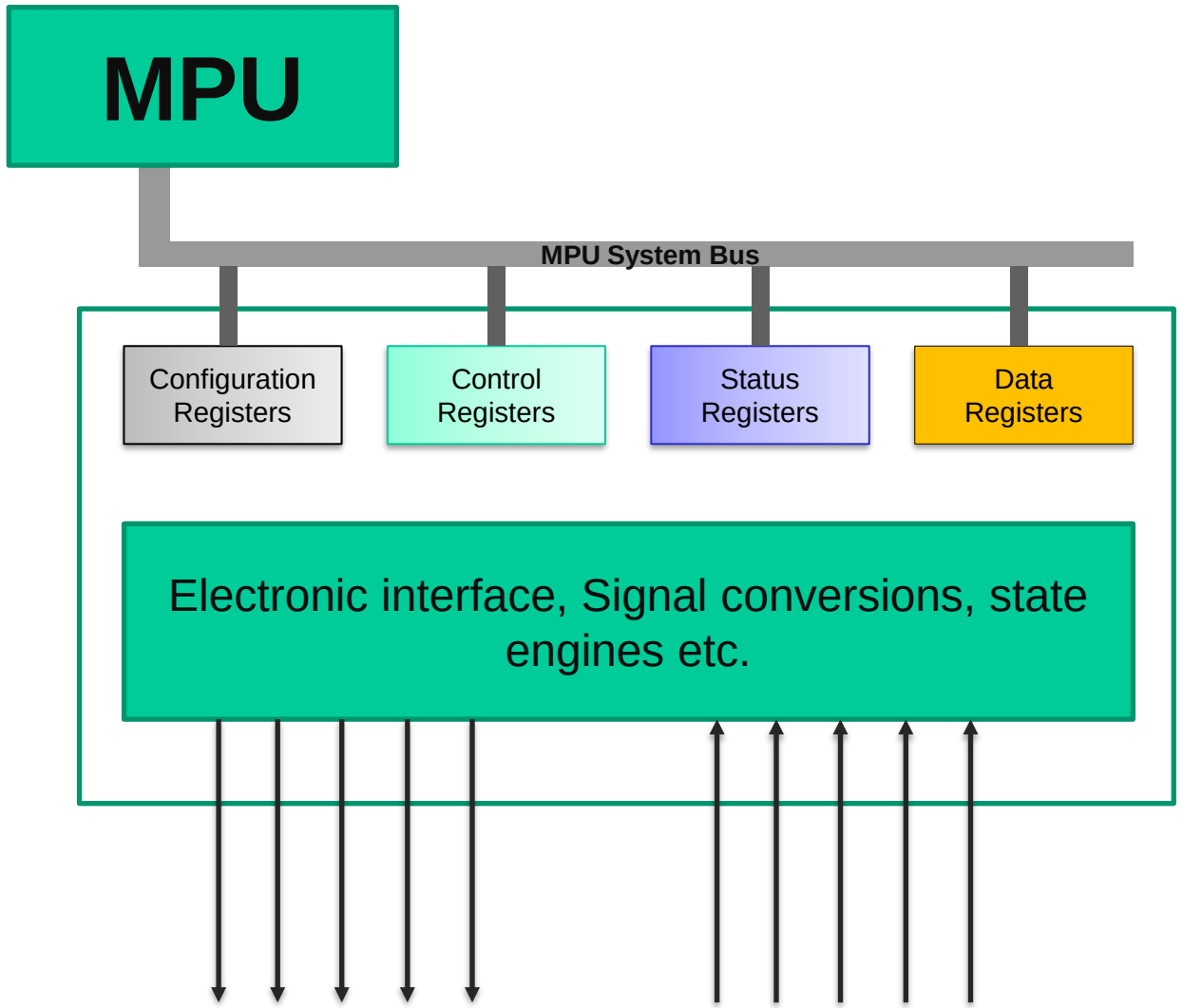


- *Arduino platform hides most of the internal configurations that are required to run the blink program*
  - *Configuring the port to drive the LED*
  - *Changing the port's output state*
  - *Creating the delay*

Peripheral devices and types

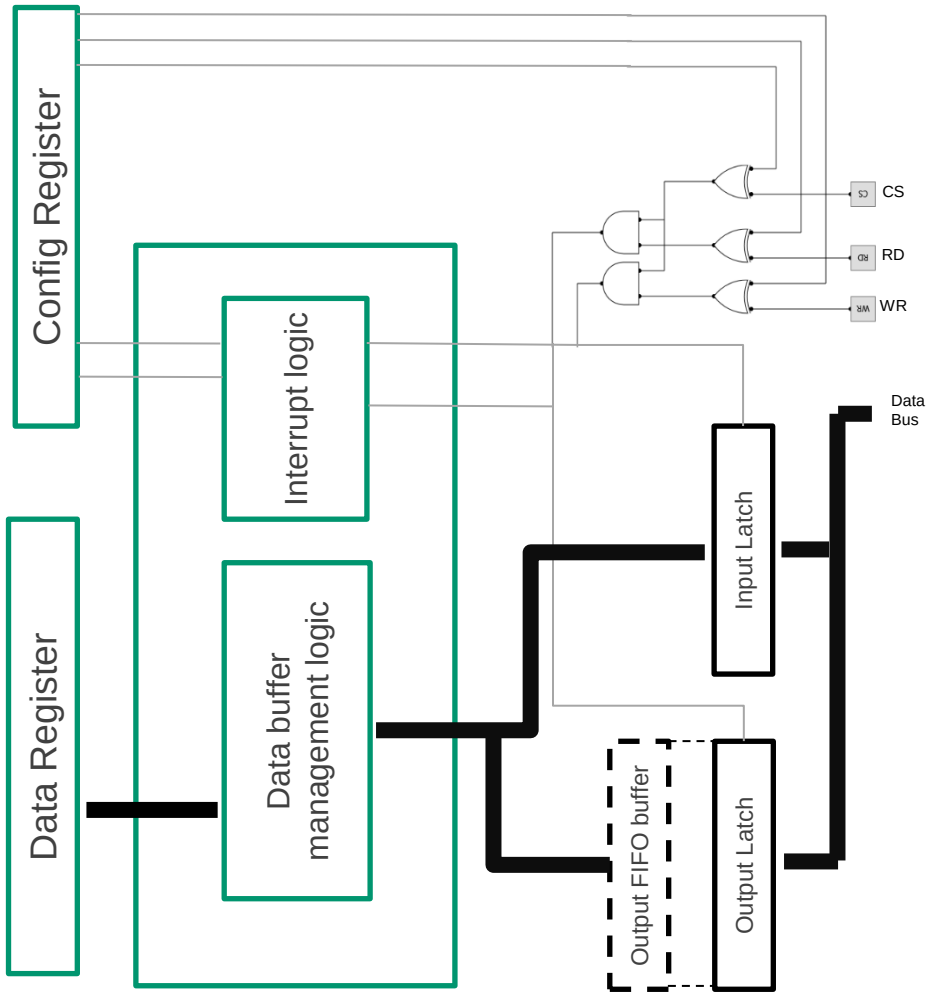
- *Peripheral devices extends the hardware and functional capabilities of the MPU in a microcontroller by providing interfaces that can talk directly with the external world*
- *They can provide different functions*
  - *Digital, Analogy Input / output modules*
  - *Communication modules*
  - *Processor extensions*
  - *Supervisory / Management interfaces*
- *They provide the interface between the bus architecture of the MPU and the external world*

Pheriperal interfaces: The general structure



- *A peripheral module in general communicate with the MPU via a set of registers*
  - *These registers are mapped onto specific IO port addresses of the MPU bus*
- *Four main types of registers*
  - *Configuration registers*
    - *Typically used to provide initial configuration of the peripheral interface*
  - *Control (Command) registers*
    - *Used to send instruction on specific tasks and initiate functional operations*
  - *Status registers*
    - *Read operational status of the peripheral device including any error conditions*
  - *Data registers*
    - *Used to transfer data between the MPU and peripheral device*

Parallel Slave Port

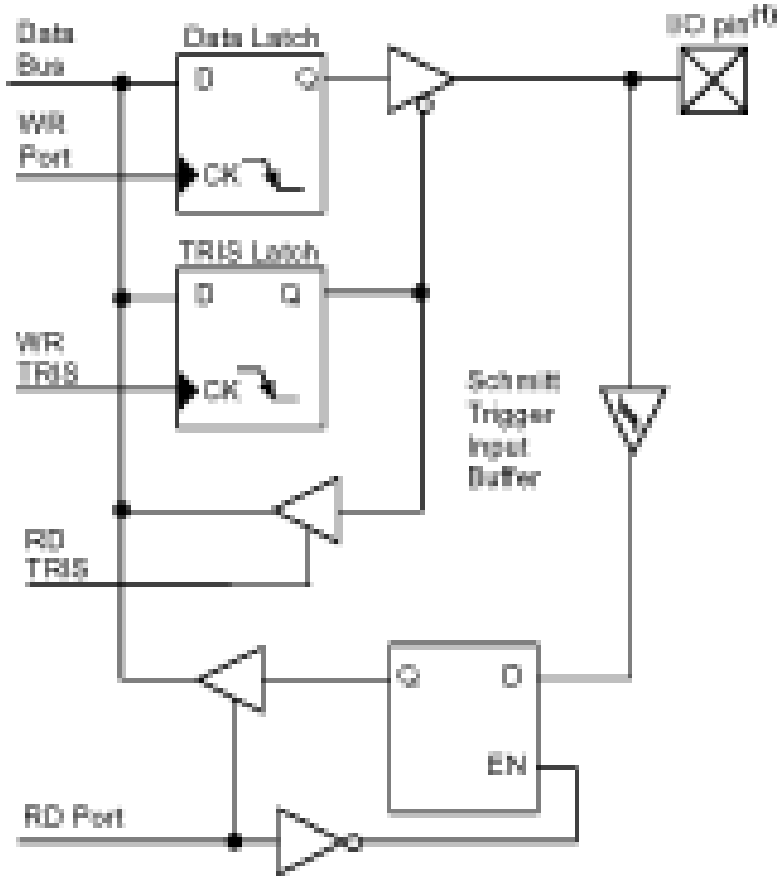


- *Used to interface with a microprocessor / system bus of an external device*
- *Compatible with control and data protocols of the external bus*
- *Configuration register provides initial setup such as enabling interrupts, polarity selection of the data latch, etc.*
- *Data output is through the write buffer register (which is usually a FIFO buffer), while read register provides the data input path*
- *Status register shows the condition of the read and write buffers*

Bit Addressable Digital IO Ports

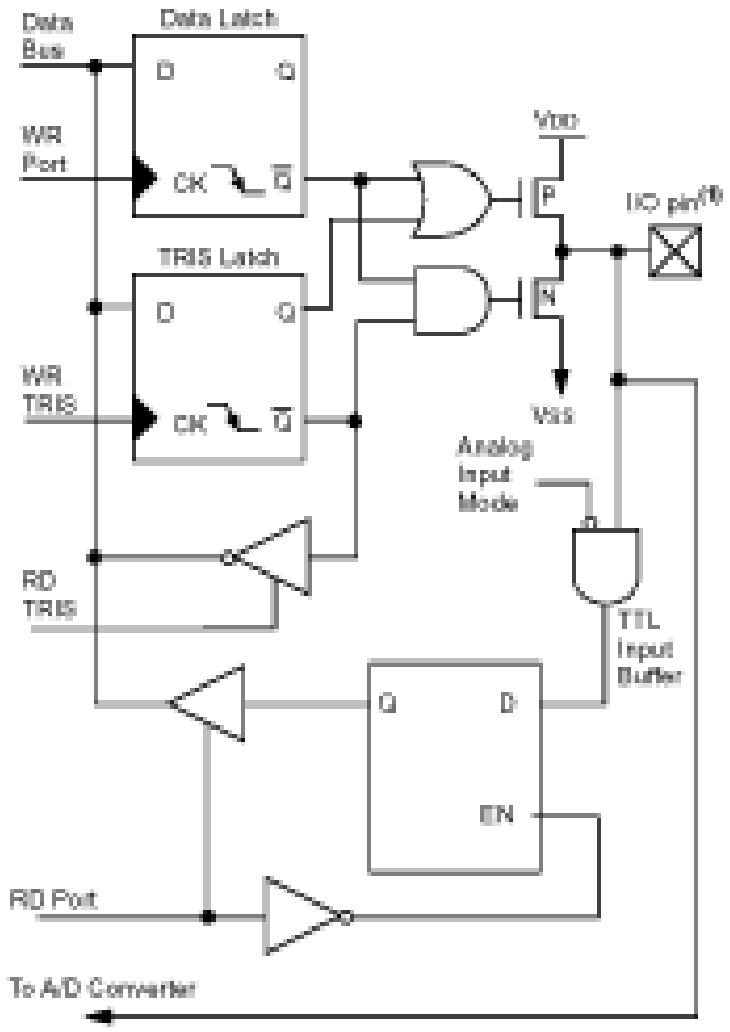
- *Bit addressable IO ports provide direct access to individual pins of the microcontroller via their respective control / data registers*
- *At hardware level these pins can provide different functions and capabilities, often controlled by the configuration register*
  - *Simple digital input / output with data latching*
  - *Combined analog / digital IO*
  - *Additional physical layer properties such as*
    - *Resister Pull-up / Pull-down*
    - *Open collector (drain) with tri-state outputs*
    - *Schematic trigger (hysteresis) inputs*

Bit Addressable Digital IO Ports



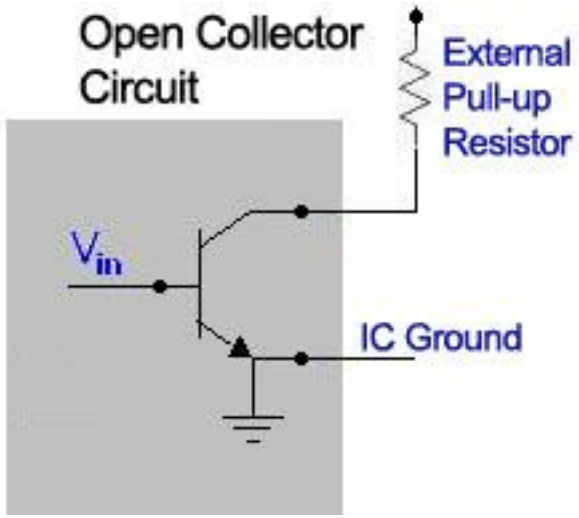
- *Simple Bit-addressable digital IO port in PIC16XXX series*
- *Configured via TRIS register that determine whether the individual bit act as an Input pin or as a latched output pin*
  - *Input pin when TRIS is true*
- *Has Schmitt trigger buffers that implement a hysteresis band to support noisy inputs*
- *Input can be latched or in transparent mode via an external **RDPort** signal*
- *The same configuration is repeated for all 8 bits in the IO port (PORTD)*

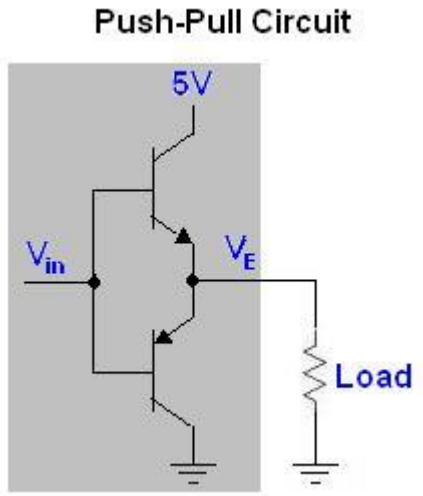
Bit Addressable Digital IO Ports



- *Digital bit addressable port with high-current capability and analog input support (PORTA in PIC16XXX)*
- *Configured via two registers*
  - *TRIS for direction control*
  - *Analog Input Mode – enable analog input*
- *Two MOSFET at the output stage allow high current driving and sinking*
- *Selecting Analog input mode disable digital reading*
  - *Pin must be in input mode to support analog input mode*
  - *Separate AtoD converter (not shown in the schematic) is required*

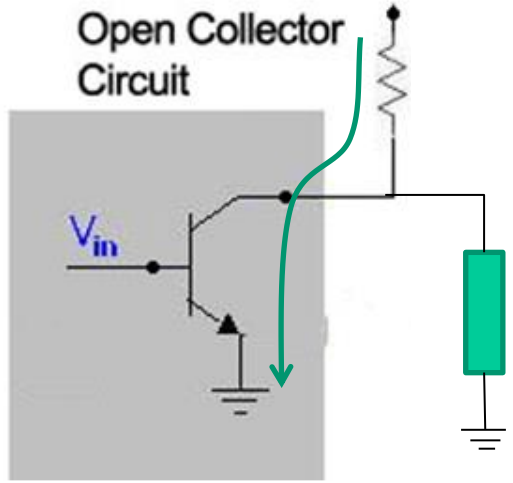
Digital IO – Push-Pull Vs Open outputs



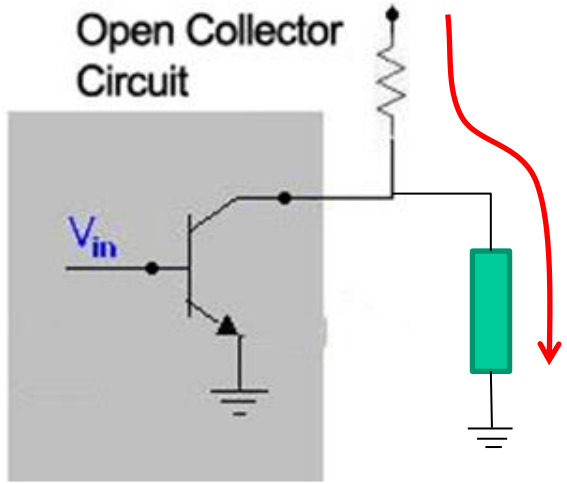


- *An open (collector / drain) configuration provides active driving only on one logic state*
- *External pull-up resisters are needed to drive the load when the device output is on “Open” state*
  - *Loading on the resister must be carefully considered to ensure correct logic state at the output*

Digital IO – Push-Pull Vs Open outputs

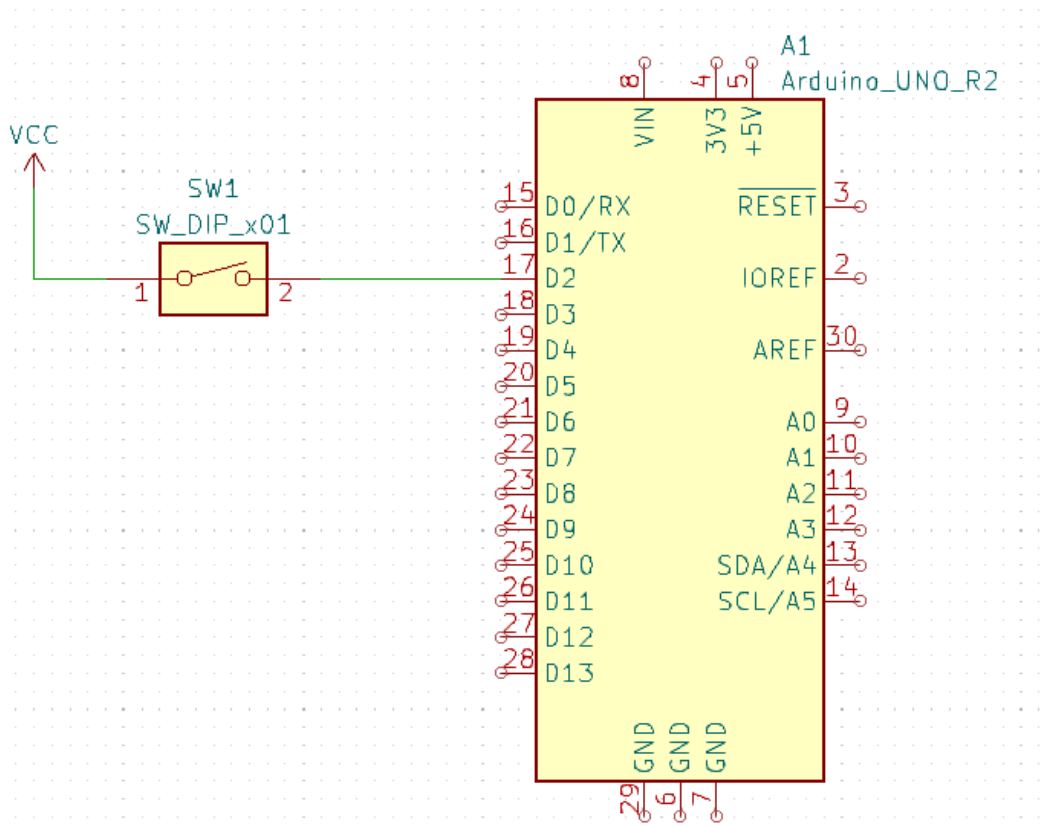


When the output is at Logic –Low state the load voltage is clamped at Vce of the output transistor. The external pull-up resister act as a current limiter



When the output is at Logic–High state current flows through the load. Output voltage is determined by potential division between the load and the external pull-up resister. Care must be taken to maintain output voltage at logic high level.

Reading one bit input



- *How would you expect the output to be?*
- *Does it behaves as expected?*

sketch_apr21a | Arduino IDE 2.1.0

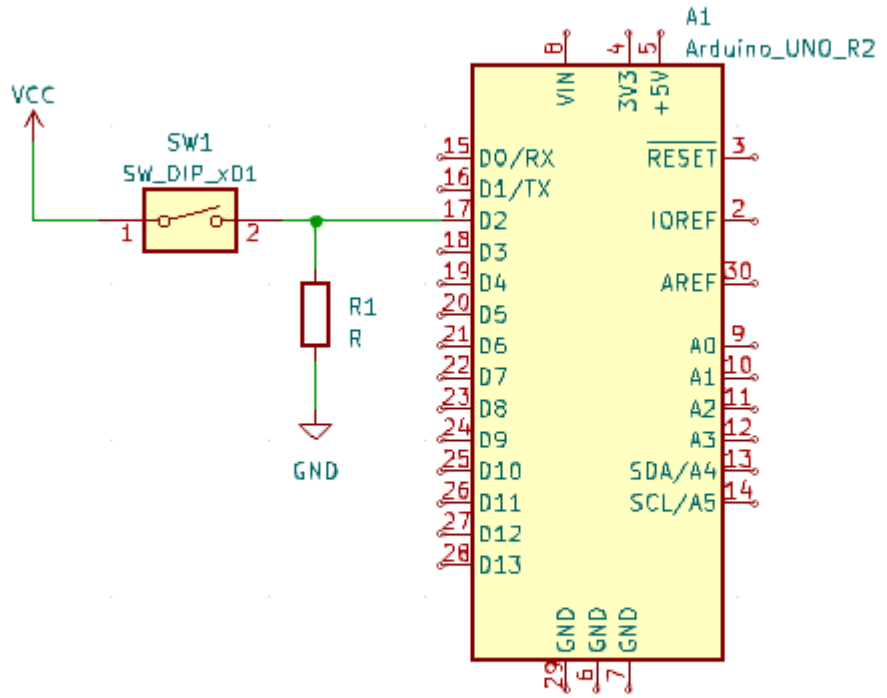
File Edit Sketch Tools Help

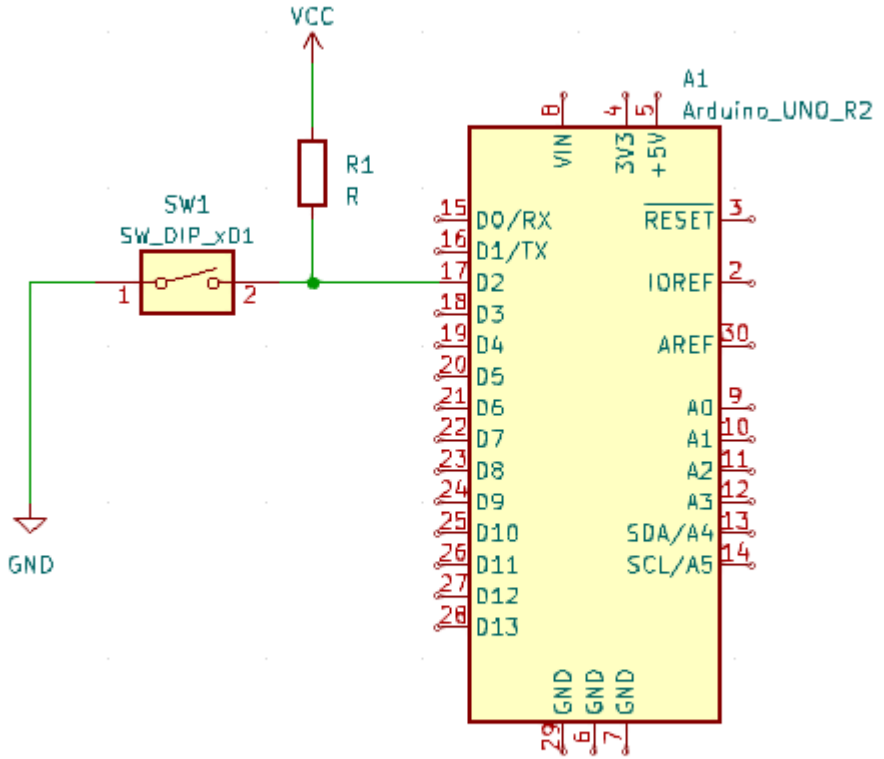
Arduino Uno

sketch_apr21a.ino

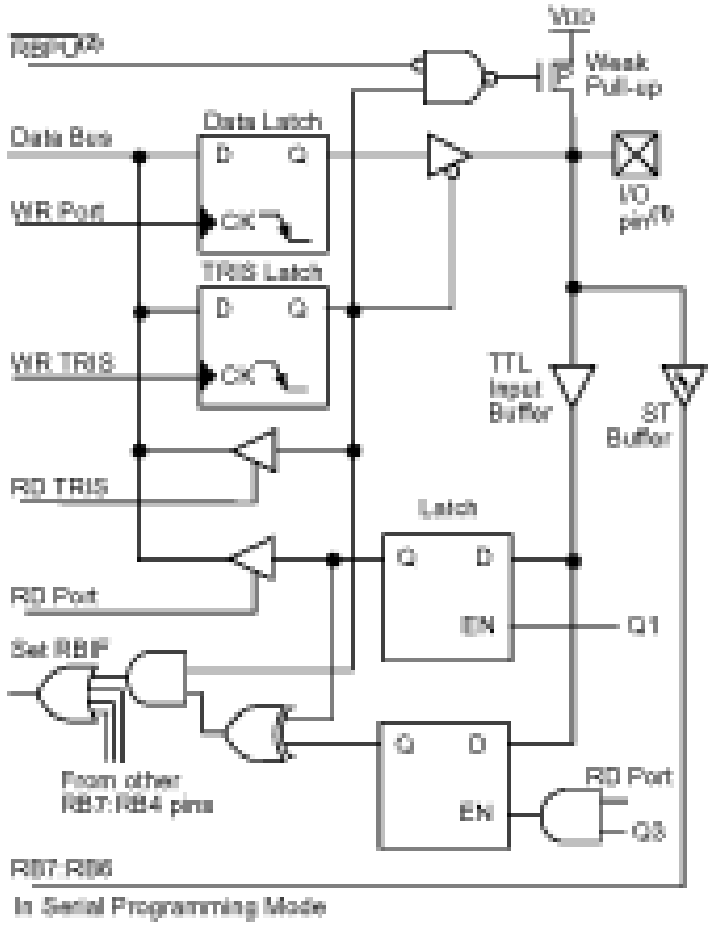
```
1  void setup() {
2    pinMode(2,INPUT);
3    pinMode(13, OUTPUT);  // Internal LED pin
4  }
5
6  void loop() {
7    int pin = digitalRead(2);
8    if (pin==1)
9      digitalWrite(13,HIGH);
10   else
11     digitalWrite(13,LOW);
12   delay(1000);
13 }
14
```

Reading one bit input: Using pull-up/Pull-Down





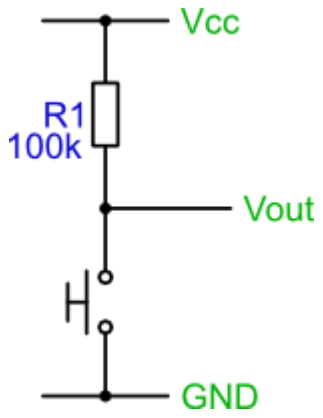
Bit Addressable Digital IO Ports: Configuration

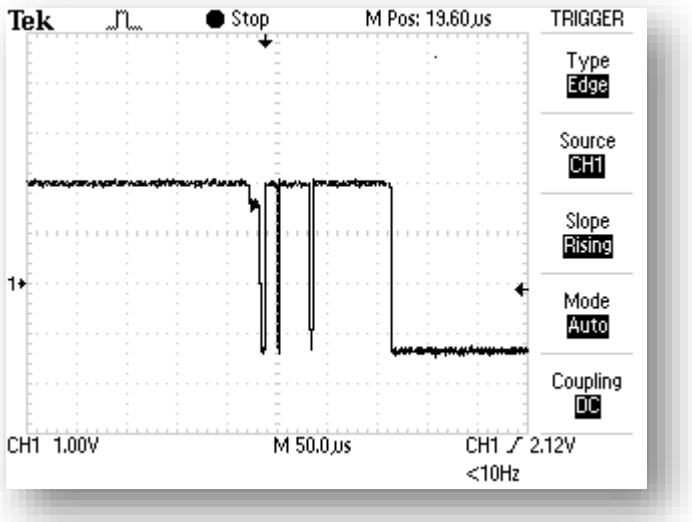


- *Digital IO port with open collector (drain) outputs, programmable pull up resisters and interrupt capability (PORTB in PIC16XXX)*
- *Configured via*
  - *TRIS register*
    - *Set pin direction (input , output)*
  - *PULLUP*
    - *Enable / Disable internal weak pull-up resisters*
  - *INTCONF*
    - *Enable interrupt on pin change*
- *Open collector (drain outputs) with low current sink capabilities*
- *Also support schimmitt trigger inputs to other peripheral devices when not used for digital IO*

Bouncing and De-bouncing ccts

- *Pressing or releasing a mechanical switch in general does not produce a clean edge in the output waveform*
  - *Digital input may interpret this as multiple edges*
  - *Referred to a “Bouncing” on the input*





A Toggle button?

LED_Toggle_Cont | Arduino IDE 2.1.0

File Edit Sketch Tools Help

Arduino Uno

LED_Toggle_Cont.ino

```
1  void setup() {
2    pinMode(2,INPUT);
3    pinMode(13, OUTPUT);  // Internal LED pin
4  }
5
6  bool LED_On = false;
7
8  void loop() {
9    int pin = digitalRead(2);
10   if (pin==1)
11     LED_On = !LED_On;
12
13   if (LED_On)
14       digitalWrite(13,HIGH);
15     else
16       digitalWrite(13,LOW);
17 }
18
```

What's wrong here…?

LED_Toggle_State | Arduino IDE 2.1.0

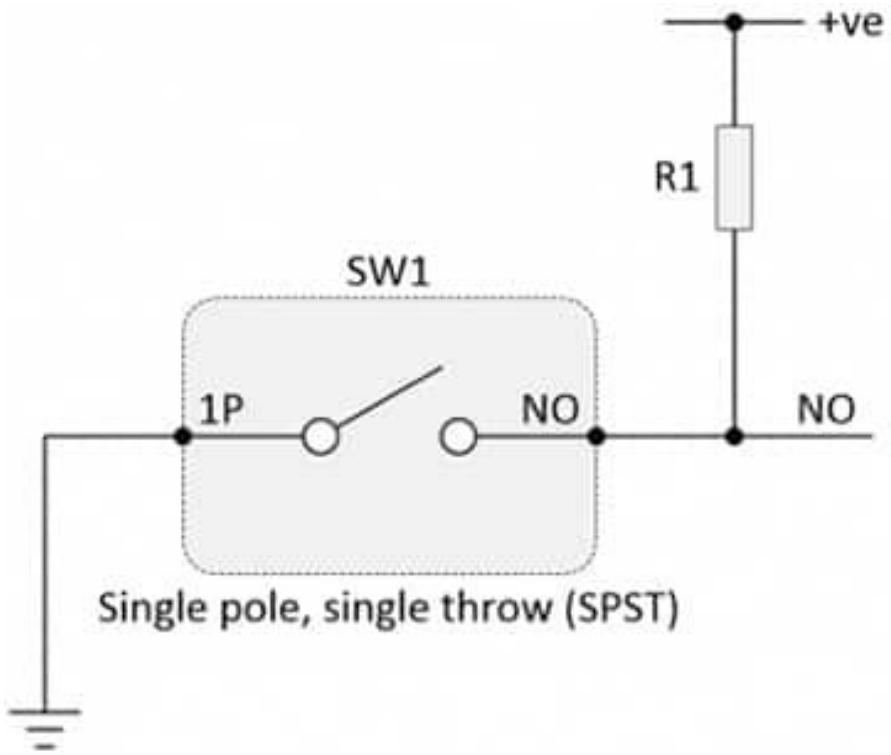
File Edit Sketch Tools Help

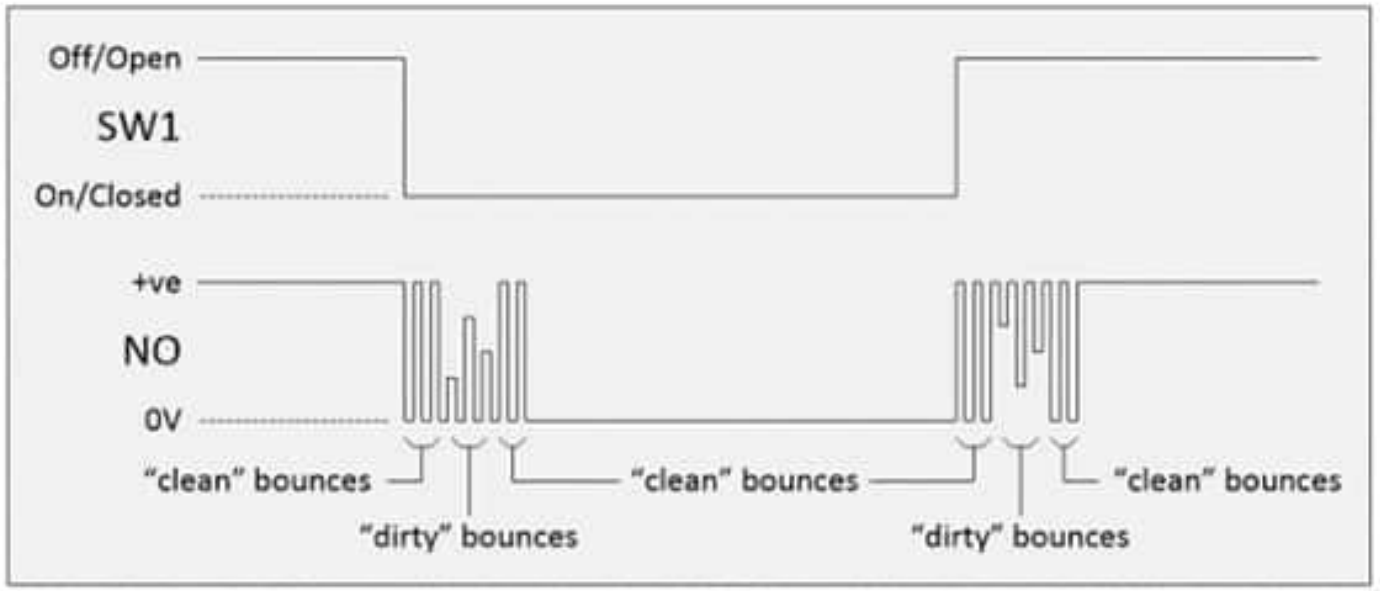
Arduino Uno

LED_Toggle_State.ino

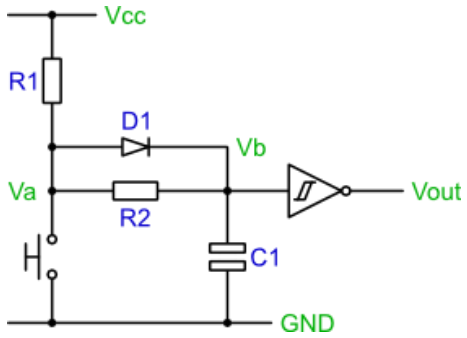
```
1  void setup() {
2    pinMode(2,INPUT);
3    pinMode(13, OUTPUT);  // Internal LED pin
4  }
5
6  bool LED_On = false;
7  bool Is_Btn_Press = false;
8
9  void loop() {
10   int pin = digitalRead(2);
11   if (pin==1) {
12     if (!Is_Btn_Press){   // rising edge
13       LED_On = !LED_On;
14       Is_Btn_Press = true;
15     }
16   } else {
17     Is_Btn_Press=false;  //Button released
18   }
19
20   if (LED_On)
21       digitalWrite(13,HIGH);
22     else
23       digitalWrite(13,LOW);
24 }
25
```

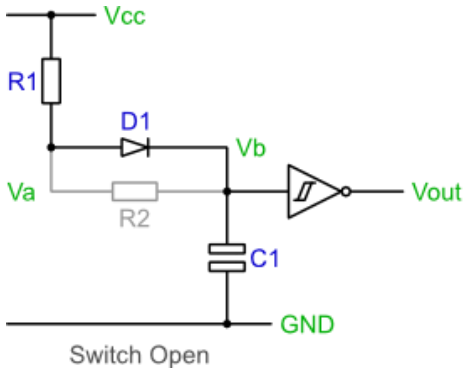
Switch bouncing and Debouncing….

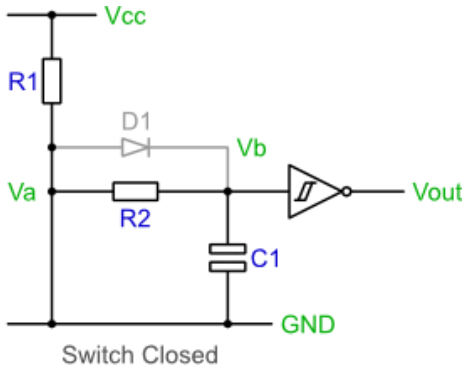




De-bouncing inputs







- *Switch Open*
  - *The capacitor C1 will charge via R1 and D1.*
  - *In time, C1 will charge and Vb will reach within 0.7V of Vcc.*
  - *Therefore the output of the inverting Schmitt trigger will be a logic 0.*

- *Switch Close*
  - *The capacitor will discharge via R2.*
  - *In time, C1 will discharge and Vb will reach 0V.*
  - *Therefore the output of the inverting Schmitt trigger will be a logic 1.*

Software de-bouncing

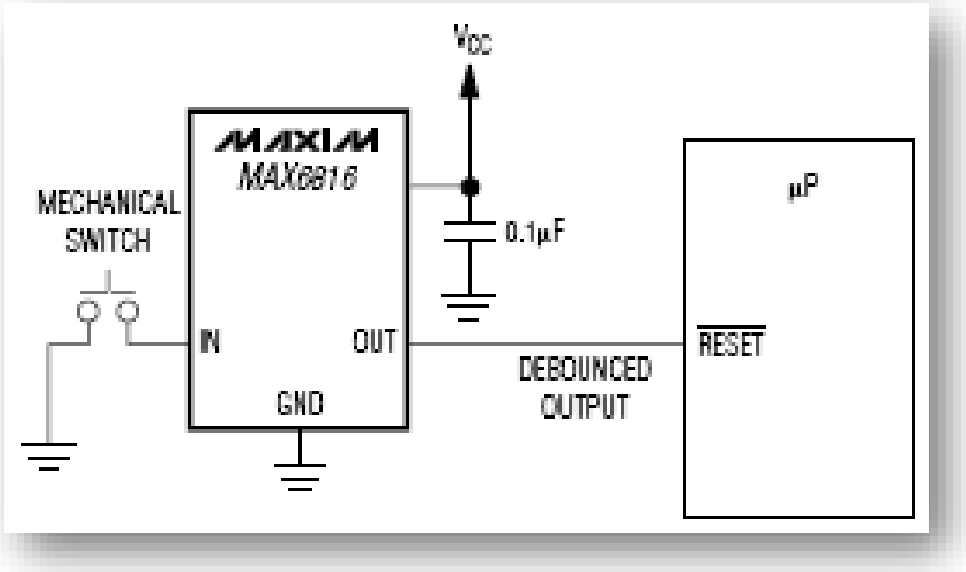
COUNTER METHOD

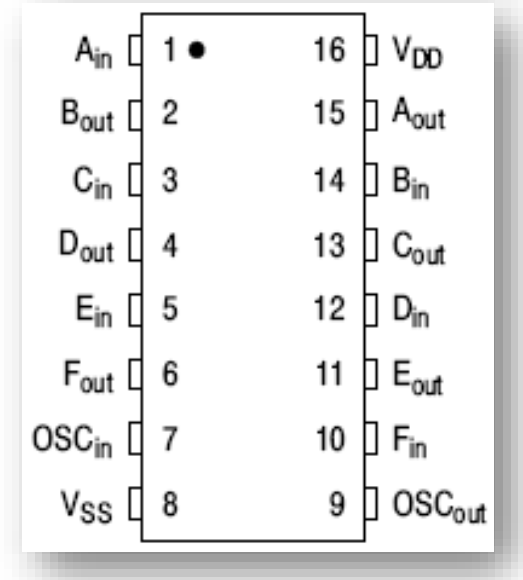
- *Setup a counter variable, initialize to zero.*
- *Setup a regular sampling event, perhaps using a timer. Use a period of about 1ms.*
- *On a sample event:*
- *if switch signal is high then*
- *Reset the counter variable to zero*
- *Set internal switch state to released*
- *else*
- *Increment the counter variable to a*
- *maximum of 10*
- *end if*
- *if counter=10 then*
- *Set internal switch state to pressed*
- *end if*

SHIFT REGISTER METHOD

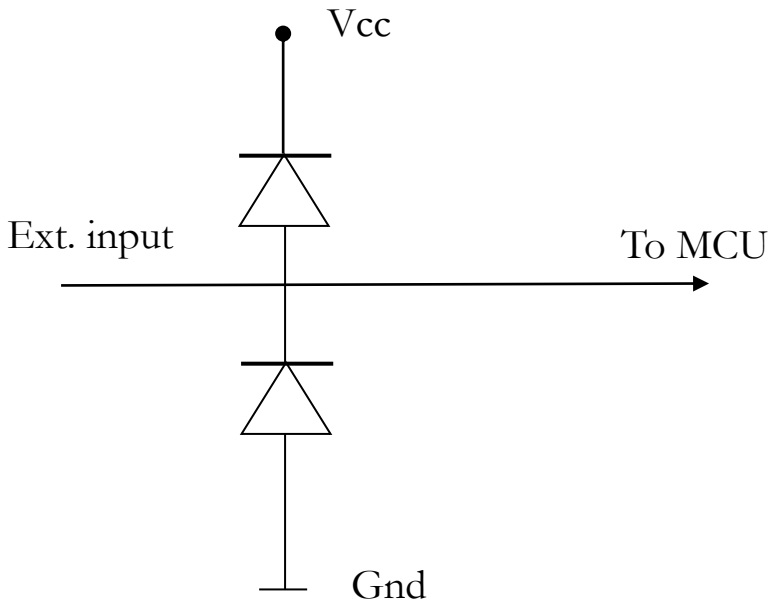
- *Setup a variable to act as a shift register, initialise it to xFF.*
- *Setup a regular sampling event, perhaps using a timer. Use a period of about 1ms.*
- *On a sample event:*
- *Shift the variable towards the most*
- *significant bit*
- *Set the least significant bit to the*
- *current switch value*
- *if shift register val=0 then*
- *Set internal switch state to pressed*
- *else*
- *Set internal switch state to released*
- *end if*

Specialized de-bouncer ICs





Protecting inputs



- *Clamping prevent the MCU input from over voltages / reverse polarities*